

Análisis de Código con ESLint

GUÍA DE CONCEPTOS DETALLADOS - UNIDAD 4

1. ¿Qué es ESLint?

ESLint es una herramienta de **linting** para JavaScript que realiza un análisis estático del código. Su objetivo principal es detectar y reportar problemas rápidamente antes de que el código se ejecute, asegurando la calidad y consistencia del proyecto.

¿Para qué sirve exactamente?

- Encontrar errores de sintaxis y patrones problemáticos (ej. variables declaradas y no usadas).
- Aplicar y forzar estándares de codificación en todo el equipo de desarrollo.
- Mantener la consistencia visual y estructural (espacios, comillas, punto y coma).

2. Instalación: NPM vs NPX

ESLint es una herramienta de desarrollo, lo que significa que no es necesaria para que la aplicación funcione en producción, solo la necesitan los programadores. Por eso, se instala como dependencia de desarrollo.

Comando	Descripción y Uso Recomendado
<code>npm install eslint --save-dev</code>	RECOMENDADO. Instala ESLint localmente y lo registra en el <code>package.json</code> bajo <code>devDependencies</code> (También se puede usar <code>-D</code>).
<code>npm install -g eslint</code>	Instalación global. No recomendado para dependencias de proyecto, ya que puede generar conflictos de versiones entre diferentes proyectos.

```
npx eslint --  
init
```

NPX ejecuta paquetes sin necesidad de instalarlos globalmente. Es ideal para inicializar la configuración de ESLint de forma limpia.

3. El Archivo de Configuración (.eslintrc)

Una vez inicializado, ESLint crea un archivo (generalmente `.eslintrc.js` o `.eslintrc.json`) que dicta cómo debe comportarse el linter. Sus propiedades clave son:

- **env** : Define dónde se va a ejecutar el código (ej. `browser: true` para frontend, `node: true` para backend). Evita que el linter marque como error palabras globales como ``window`` o ``process``.
- **extends** : Permite heredar reglas predefinidas por la comunidad o empresas. Ej: `'eslint:recommended'` o `'airbnb'`.
- **parserOptions** : Define qué versión de JavaScript se está utilizando (ej. `ecmaVersion: 'latest'`).
- **rules** : El corazón de la configuración. Aquí se sobrescriben o añaden reglas específicas para el proyecto.

4. Sistema de Reglas y Severidad

Cada regla en ESLint se puede configurar con tres niveles de severidad (usando números o palabras):

Nivel	Valor numérico	Comportamiento
"off"	0	Apaga la regla por completo. El linter la ignora.
"warn"	1	Muestra una advertencia. Útil para malas prácticas que no rompen el código. No detiene procesos de CI/CD.
"error"	2	Marca un error crítico. Detiene compilaciones o pipelines de Integración Continua (CI).

```
// Ejemplo de bloque 'rules' en .eslintrc.js
"rules": {
  "semi": ["error", "always"],    // Exige punto y coma
  obligatoriamente
  "quotes": ["error", "single"],  // Exige comillas simples
  "no-console": ["warn"],        // Advierte si dejas un
  console.log
  "no-unused-vars": ["error"]     // Error si declaras una
  variable y no la usas
}
```

5. Ignorar Reglas (Excepciones)

A veces, necesitamos romper una regla por un motivo justificado. ESLint nos permite hacer excepciones de tres formas:

- **Línea específica:**

Agregando `// eslint-disable-next-line nombre-regla` justo arriba de la línea problemática.

- **Bloque de código:** Envolviendo el código entre `/* eslint-disable */` y `/* eslint-enable */`.

- **Archivos enteros:** Creando un archivo `.eslintignore` (similar a `.gitignore`) para decirle al linter que ignore carpetas como `node_modules/` o `dist/`.

6. Ecosistema y CI/CD

Plugins y Frameworks

ESLint no se limita a JavaScript puro (Vanilla JS). Mediante la propiedad **plugins**, se puede extender para entender la sintaxis de **React** (JSX), **Vue.js**, o **TypeScript** (usando un parser especial como `@typescript-eslint/parser`).

Integración Continua (CI/CD)

En entornos profesionales, ESLint se ejecuta automáticamente en plataformas como GitHub Actions antes de permitir que un código se fusione (Merge) a la rama principal. Si se configura un script en el `package.json` como `"lint": "eslint ."`, el servidor ejecutará este comando. Si detecta alguna regla con nivel "error" (2), **rechazará el código**, garantizando que nunca llegue código sucio a producción.